

Nexus

LLM Automated Information Retrieval System

*Transforming Enterprise Knowledge Management Through
Multi-Agent Intelligence, GraphRAG, and Local-First Sovereignty*

1. Executive Summary

Nexus is an enterprise-grade, locally-sovereign knowledge retrieval platform that fundamentally rethinks how organisations extract intelligence from their document repositories. Built on a three-pillar architecture — Multi-Agent Swarm intelligence, GraphRAG powered by Neo4j, and FAISS-based semantic indexing — Nexus delivers contextually rich, multi-hop answers that traditional Retrieval-Augmented Generation (RAG) systems are incapable of providing.

At its core, Nexus treats organisational knowledge not as a flat collection of text chunks but as a living, interconnected knowledge graph. Every document ingested into the platform is decomposed into entities and typed relationships that are woven into this graph. When a user poses a question, a coordinated swarm of specialised agents simultaneously searches the semantic vector index and traverses the knowledge graph, fusing results before a synthesis agent composes a grounded, fully-cited response.

Critically, the entire pipeline — from document upload and OCR through human-in-the-loop approval, entity extraction, and inference — runs entirely on-premises inside Docker containers orchestrated by Kubernetes. No document, embedding, or query ever leaves the organisational perimeter. This architecture makes Nexus uniquely positioned to serve regulated industries such as financial services, healthcare, pharmaceuticals, defence, and legal — sectors where data sovereignty is non-negotiable.

Key capabilities at a glance: multi-hop graph reasoning across millions of documents; human-in-the-loop governance with Slack-based approval gates; sub-millisecond semantic retrieval via FAISS; LLM-response caching via GPTCache and Redis; automated document lifecycle orchestration via n8n; full audit trails; and compliance-ready controls for GDPR, HIPAA, SOC 2, and the NIST Cybersecurity Framework.

2. Introduction: The Enterprise Knowledge Crisis

Large organisations today are simultaneously data-rich and insight-poor. According to industry estimates, knowledge workers spend between 20 and 30 percent of their working week searching for information that already exists somewhere inside their organisation. Reports, policy documents, engineering specifications, customer contracts, research papers, Slack conversations, and meeting transcripts accumulate faster than any human team can process — yet the intelligence locked inside them often remains inaccessible at the moment of decision.

The emergence of Large Language Models (LLMs) and Retrieval-Augmented Generation offered a promising solution: give the model access to a document corpus and let it answer questions in natural language. Early adopters quickly discovered, however, that standard RAG architectures have a fundamental ceiling. They work well for simple, self-contained queries ("What is our refund policy?") but fail under the weight of complex, multi-document questions that require understanding relationships, causality, and organisational context. A question such as "Which of our Tier-1 suppliers are exposed to the same geopolitical risk as Supplier X, and what alternative sources do we have?" cannot be answered by retrieving the most similar paragraph — it requires traversing a graph of relationships across dozens of documents simultaneously.

Nexus was designed from first principles to close this gap. Rather than bolting retrieval onto an LLM, it builds a rich semantic and structural representation of organisational knowledge and deploys a swarm of reasoning agents to navigate it. The result is a system that answers the questions that matter most — the complex, multi-hop, relationship-aware ones — while maintaining the security and sovereignty guarantees that enterprise deployments demand.

2.1 Why Traditional RAG Falls Short

Standard RAG systems follow a three-step loop: embed the query, retrieve the top-K most semantically similar document chunks from a vector index, then pass those chunks as context to an LLM. This approach has four critical failure modes that become increasingly apparent as query complexity grows.

- **Context Isolation** — each document chunk is treated as an independent unit. Relationships between entities mentioned across different documents are completely invisible to the retriever. The system cannot answer "How does Event A in Document 1 relate to Entity B in Document 47?"
- **Relationship Blindness** — semantic similarity measures whether two pieces of text sound alike, not whether they are logically connected. "Supplier X is certified by ISO 9001" and "ISO 9001 requires corrective action plans" may be retrieved together, but the implication — that Supplier X must have a corrective action plan — requires traversing a relationship the vector index does not capture.
- **Context Window Saturation** — when a question requires synthesising information from many sources, the top-K retrieved chunks rapidly fill the LLM context window with redundant or tangentially related content, crowding out the genuinely relevant evidence.
- **Temporal and Causal Blindness** — standard RAG has no mechanism to represent "before", "caused by", "resulted in", or "superseded by" relationships. Time-sensitive operational queries — such as understanding how a regulatory change in Q3 affects a contract signed in Q1 — require a structured temporal graph that vector indexes cannot provide.

2.2 The Security and Sovereignty Imperative

Beyond retrieval accuracy, enterprises face a second constraint that most commercial AI platforms ignore: data sovereignty. Sending sensitive corporate documents to a third-party cloud API is not merely a technical decision — it carries significant legal, regulatory, and reputational risk. For organisations operating under GDPR, HIPAA, FedRAMP, or sector-specific frameworks such as PCI-DSS or ITAR, routing confidential data through external inference

endpoints may constitute a regulatory violation regardless of contractual data-processing agreements.

- **Data Exposure Risk** — confidential financial models, patient records, trade secrets, and M&A documents must never transit external networks. Once data leaves the perimeter, the organisation loses meaningful control over how it is stored, processed, or retained.
- **Audit and Explainability Gaps** — cloud-based LLM APIs are black boxes. Organisations subject to audit cannot demonstrate precisely which documents informed a given answer, nor can they prove that no hallucinated information was introduced.
- **Vendor Lock-in and Latency** — dependence on a cloud AI vendor introduces operational risk. Service outages, pricing changes, model deprecations, and API rate limits can all disrupt business-critical workflows. Network round-trips also add latency that local inference eliminates.
- **Prompt Injection and Data Poisoning** — routing organisational data through shared cloud infrastructure exposes organisations to cross-tenant contamination risks and adversarial prompt injection attacks that are fundamentally mitigated by air-gapped local deployment.

Nexus addresses every one of these concerns by design. The entire inference and retrieval stack runs inside the customer's own infrastructure — on bare metal, private cloud VMs, or an on-premises Kubernetes cluster — with no mandatory external connectivity.

2.2 The Nexus Vision: Knowledge as a Graph

The foundational insight behind Nexus is that organisational knowledge is not a collection of independent text fragments — it is a network. People belong to teams, teams own projects, projects depend on suppliers, suppliers operate in markets, markets are shaped by regulations, and regulations affect contracts. Every document in an organisation's corpus is a window onto a small portion of this larger network. Nexus makes that network explicit, queryable, and traversable.

By modelling knowledge as a property graph in Neo4j, Nexus transforms question-answering from a similarity-search problem into a graph-traversal problem. This unlocks a qualitatively different class of queries: multi-hop reasoning, path analysis, subgraph matching, and community detection. Combined with FAISS semantic search for rapid candidate retrieval and a swarm of specialised LLM agents for reasoning and synthesis, Nexus delivers answers with a depth and accuracy that no flat-RAG system can match.

3. System Architecture & Design

Nexus is architected as a set of loosely coupled, containerised microservices grouped into six logical layers. This layered design allows each component to be scaled, upgraded, or replaced independently without disrupting the rest of the system. The architecture diagram below provides a high-level view of all layers and the data flows that connect them.

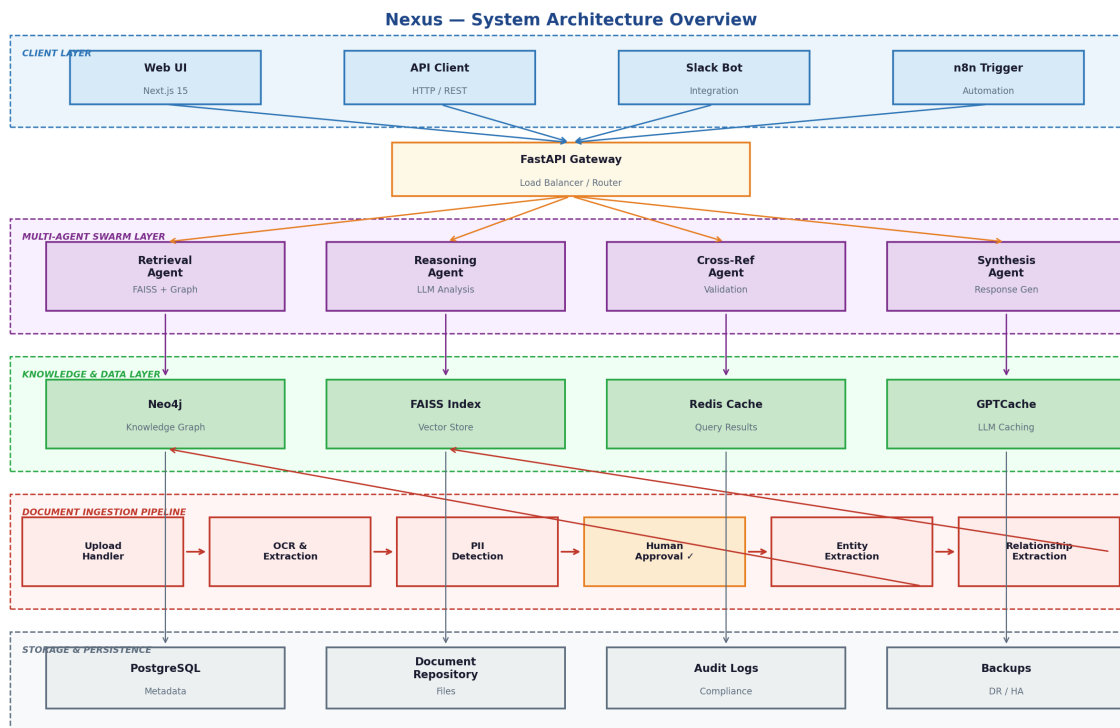


Figure 1 — Nexus System Architecture: six layers from client interfaces through persistent storage

3.1 Layer-by-Layer Breakdown

Client Layer

The client layer exposes three primary access surfaces. The Web UI, built on Next.js 15 with server-side rendering, provides a full-featured interactive interface including a natural-language query console, a Neural Topology visualiser that renders the live knowledge graph, a Tactical Dashboard with real-time analytics, and a document management panel. The REST API Client surface allows external applications, scripts, and third-party integrations to programmatically query the Nexus knowledge base using standard HTTP/JSON. The Slack Bot integration enables team members to submit queries directly from Slack channels without leaving their existing workflow — the bot routes the query to the FastAPI gateway and returns a formatted, cited response as a thread reply. Finally, n8n automation triggers allow external events — such as a new file appearing in a watched folder or a webhook from a document management system — to initiate document ingestion automatically.

API Gateway Layer

All inbound traffic — whether from the web browser, API clients, Slack, or n8n — is routed through a FastAPI gateway that acts as the system's central nervous system. The gateway is responsible for authentication and authorisation (JWT tokens with role-based access control), request validation and schema enforcement, rate limiting to prevent resource exhaustion, request routing to the appropriate downstream agents or ingestion workers, and structured logging of every request for the audit trail. Because FastAPI is built on Python's asyncio

runtime, it handles large numbers of concurrent requests with minimal thread overhead, making it well-suited to the bursty, latency-sensitive traffic patterns of interactive knowledge retrieval.

Multi-Agent Swarm Layer

At the heart of Nexus is the multi-agent swarm — a coordinated ensemble of four specialised agents that collaborate to answer each query. Unlike a single monolithic LLM call, the swarm approach allows parallel execution, specialised reasoning strategies, and iterative refinement. The swarm is described in detail in Section 4.1.

Knowledge & Data Layer

This layer hosts the three core data stores that the agent swarm queries: the Neo4j property graph database containing all extracted entities and relationships; the FAISS vector index containing dense embeddings of every document chunk; and the Redis cache that stores recent query results, hot embeddings, and intermediate agent state. GPTCache sits above Redis to provide semantic-level LLM-response caching — if two queries have similar embeddings and would likely produce the same answer, GPTCache returns the cached response without invoking the LLM, dramatically reducing both latency and compute cost.

Document Ingestion Pipeline

Every document that enters Nexus passes through a ten-stage ingestion pipeline that is fully automated by n8n workflows. The pipeline extracts text, detects sensitive content, routes flagged documents for human review, extracts entities and relationships, and populates both the Neo4j graph and the FAISS index. The pipeline is described in full in Section 6.

Storage & Persistence Layer

All durable state is persisted in this layer. PostgreSQL stores operational metadata — document records, user accounts, RBAC policies, ingestion job statuses, and the complete audit log. The document repository stores original source files and their extracted text in an immutable, version-controlled store. Separate volumes hold Neo4j graph data, FAISS index shards, and full system backups. All volumes are encrypted at rest using AES-256.

4. Three-Pillar Architecture

The retrieval and reasoning capabilities of Nexus rest on three mutually reinforcing pillars. Each pillar addresses a different dimension of the knowledge retrieval problem: the agent swarm handles reasoning and coordination; GraphRAG handles structural and relational knowledge; and FAISS handles high-speed semantic proximity search. Together, they cover the full spectrum from "find the most similar text" to "traverse five relationship hops to infer an implicit answer".

4.1 Pillar 1 — Multi-Agent Swarm Intelligence

The swarm architecture is inspired by the principle that complex problems are best solved by a team of specialists rather than a single generalist. Each agent in the Nexus swarm is a purpose-built LLM pipeline with its own system prompt, tool set, and success criteria. Agents communicate through a shared context object that is progressively enriched as the query moves through the pipeline.

Agent	Role	Key Tools & Behaviours
-------	------	------------------------

Retrieval Agent	First responder — rapid candidate discovery	Embeds the query, fires parallel FAISS top-K and Neo4j entity-lookup queries, scores and deduplicates results, returns a ranked candidate set with provenance metadata.
Reasoning Agent	Deep analyst — context evaluation and gap detection	Reads the candidate set, assesses whether it contains sufficient evidence to answer the query, identifies specific knowledge gaps, and issues targeted follow-up queries to the graph or vector index. May iterate multiple times before proceeding.
Cross-Reference Agent	Fact checker — consistency and conflict resolution	Compares claims across multiple retrieved sources, flags contradictions, verifies temporal consistency (e.g., an earlier document may be superseded by a later one), and traces every claim back to its source document and page number.
Synthesis Agent	Writer — grounded, cited response generation	Assembles the verified evidence into a coherent natural-language response, inline-cites every factual claim with document title and location, and formats the output according to the requested mode (summary, table, detailed analysis, etc.).

The swarm architecture delivers four measurable advantages over a single LLM call. First, the Retrieval and Reasoning agents execute their graph and vector queries in parallel, which significantly reduces end-to-end latency compared to sequential retrieval. Second, agent specialisation means each component is optimised for its specific task — the Retrieval Agent uses lightweight embedding models for speed, while the Synthesis Agent uses a larger, more capable model for language quality. Third, the iterative refinement loop in the Reasoning Agent naturally handles multi-hop queries by issuing successive targeted queries until the knowledge gap is closed. Fourth, because each agent's contribution to the final answer is logged separately, the system produces a full audit trail showing exactly which sources informed each claim in the response.

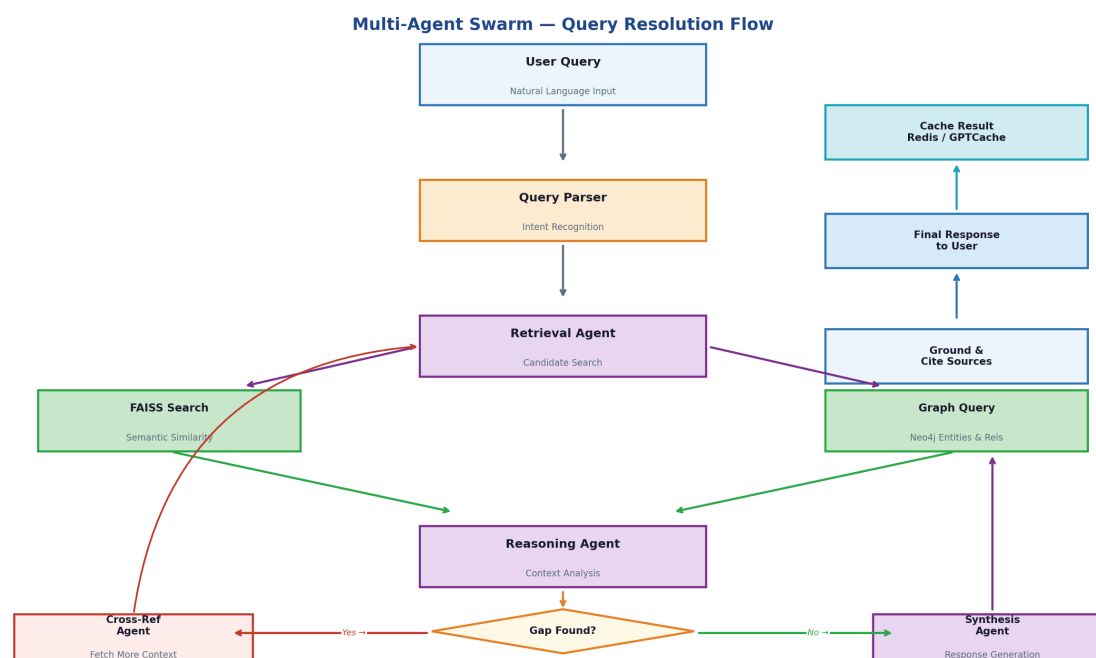


Figure 2 — Multi-Agent Swarm Query Resolution Flow: from user query through parallel retrieval, reasoning, cross-reference, and synthesis

4.2 Pillar 2 — GraphRAG & Knowledge Graph (Neo4j)

While the FAISS index answers the question "which documents sound most like my query?", the knowledge graph answers the question "how are the entities mentioned in my query related to each other and to other entities across the corpus?" This structural perspective is what enables Nexus to answer complex, multi-hop questions that defeat every flat-RAG approach.

Neo4j is the graph database at the centre of this pillar. It is a native property graph database that stores entities as labelled nodes and relationships as typed, directed edges, both of which can carry arbitrary properties. Its Cypher query language is purpose-built for expressing path traversals, pattern matches, and graph algorithms — operations that would require complex, fragile SQL joins on a relational database. Neo4j's native graph storage engine also means that multi-hop traversals are orders of magnitude faster than equivalent relational queries, because the database follows physical pointers between nodes rather than performing table scans and hash joins.

Entity Extraction in Depth

When a document passes through the ingestion pipeline, a dedicated NER module (combining a fine-tuned transformer model with rule-based post-processing) scans the text and identifies named entities. Nexus recognises eight default entity types — Person, Organisation, Product, Technology, Location, Regulation, Concept, and Event — and supports custom entity types configured per deployment. Each extracted entity is assigned a canonical identifier so that "Acme Corp", "Acme Corporation", and "Acme" in different documents are all resolved to the same graph node. This entity resolution step is critical: without it, the graph fragments into isolated islands rather than forming a connected knowledge network.

Relationship Extraction in Depth

Once entities are identified, a second LLM-powered pass extracts the semantic relationships between them. Nexus uses a zero-shot relation extraction prompt that asks the model to identify subject-predicate-object triples within a sliding context window. The extracted predicates are normalised to a controlled vocabulary of typed relationship labels — WORKS_AT, MANAGES, COMPETES_WITH, MANUFACTURES, REGULATED_BY, SUPERSEDES, CAUSED_BY, LOCATED_IN, and so on. Each relationship is stored as a directed edge with properties including the source document ID, the extraction confidence score, the page and paragraph number, and a timestamp. This provenance data allows the Cross-Reference Agent to trace every claim back to its origin.

Graph Storage, Querying, and Analytics

The resulting property graph is stored in Neo4j and made available to the agent swarm via a Cypher query interface. A typical multi-hop query might look like: "Find all organisations that supply components to direct competitors of Acme Corp, that are also regulated by the same regulatory body as Acme Corp, and return the most recent document in which each such supplier is mentioned." This query traverses four relationship types (COMPETES_WITH, SUPPLIES_TO, REGULATED_BY, MENTIONED_IN) across potentially thousands of nodes — a pattern that is trivially expressible in Cypher but computationally infeasible with flat text retrieval.

Beyond direct querying, Nexus also exposes several graph analytics capabilities through the Neural Topology dashboard: community detection to identify clusters of related entities, centrality analysis to surface the most influential nodes, shortest-path queries to find the connection chain between any two entities, and temporal graph views that animate how the knowledge graph has evolved as new documents were ingested.

Nexus — Knowledge Graph Structure & Entity Relationships

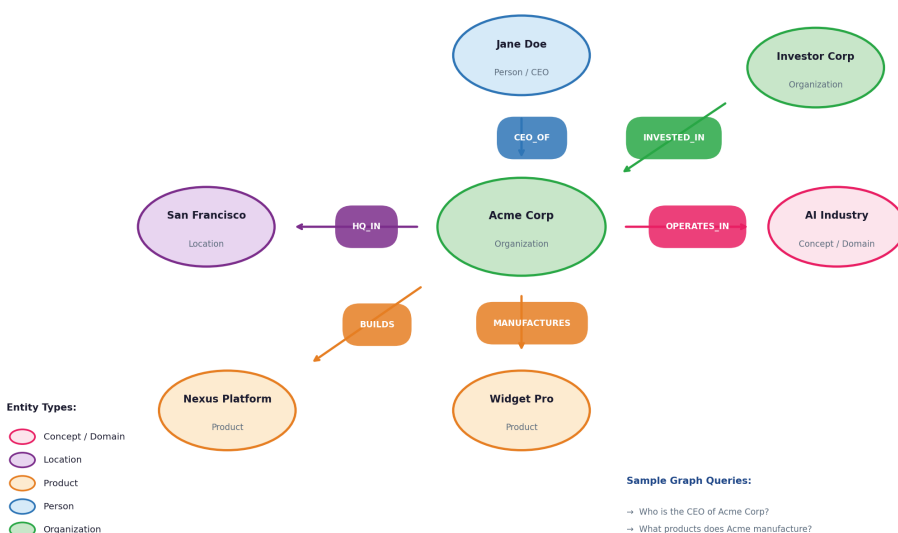


Figure 3 — Knowledge Graph Structure: entity types, typed relationship edges, and example instances with sample queries

4.3 Pillar 3 — FAISS-Powered Semantic Search

FAISS (Facebook AI Similarity Search) is a high-performance library developed by Meta AI Research for efficient similarity search over dense vector embeddings. It is the component responsible for answering "find me the text chunks most semantically similar to this query" — the initial broad retrieval step that feeds the Reasoning Agent with candidate evidence before the knowledge graph narrows and deepens the answer.

Every document ingested into Nexus is split into overlapping chunks of configurable size (default: 512 tokens with a 64-token overlap). Each chunk is passed through an embedding model (default: a locally-hosted sentence-transformer such as all-MiniLM-L6-v2 or a larger multilingual model for non-English corpora) to produce a dense 384- or 768-dimensional vector. These vectors are inserted into a FAISS IndexIVFPQ index — a product-quantised inverted-file index that dramatically reduces memory footprint while maintaining sub-millisecond search latency even at corpus sizes of hundreds of millions of chunks.

- **Speed** — FAISS can search across 100 million 768-dimensional vectors in under 10 milliseconds on a single CPU node, and under 1 millisecond with GPU acceleration.
- **Memory Efficiency** — product quantisation compresses each vector to a fraction of its original size (typically 4–16x compression) with only a small accuracy trade-off, enabling large corpora to fit in memory.
- **Local Execution** — the FAISS index runs entirely in-process within the Retrieval Agent container; there are no network calls to an external vector database service.
- **Hybrid Re-ranking** — raw FAISS scores (based on cosine similarity or L2 distance) are combined with graph-proximity scores from Neo4j in a weighted fusion step, ensuring that structurally relevant results are not penalised for superficial lexical differences.
- **Incremental Updates** — new document chunks are inserted into the FAISS index in near-real-time as ingestion completes, without requiring a full index rebuild.

5. Data Flow & Query Processing

5.1 The Hybrid Retrieval Model

Every query submitted to Nexus passes through a deterministic four-stage pipeline that balances retrieval breadth (FAISS) with retrieval depth (knowledge graph), orchestrated by the agent swarm. The diagram below provides a detailed view of each stage and the data transformations within it.

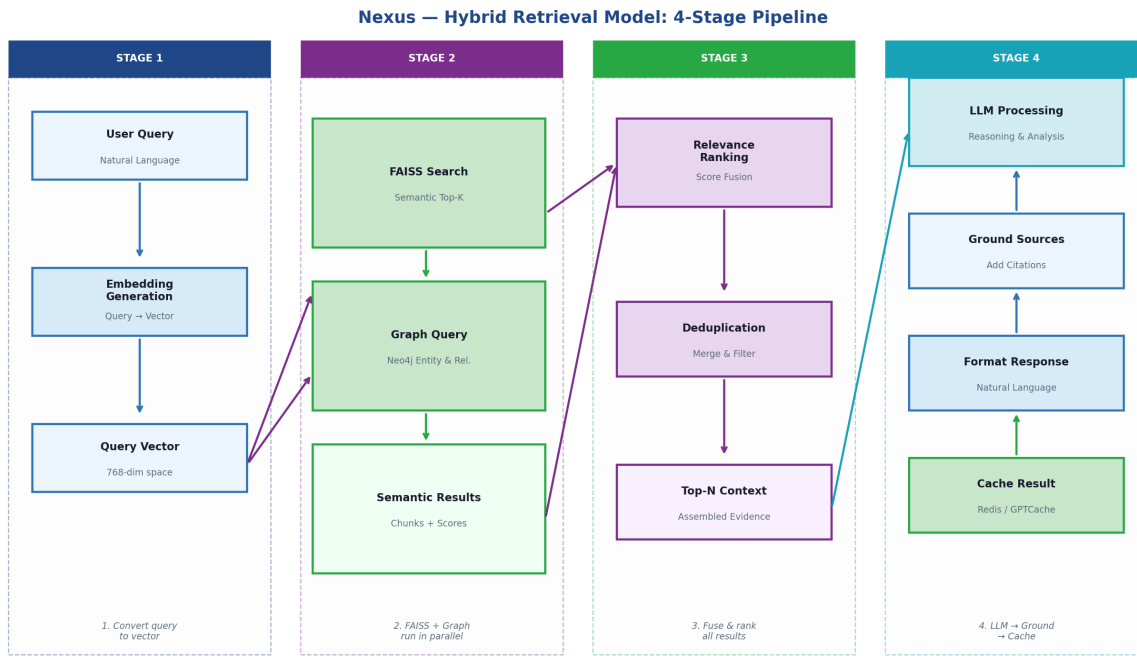


Figure 4 — Hybrid Retrieval Model: four-stage pipeline from query embedding through parallel retrieval, result fusion, LLM reasoning, and cached response delivery

Stage 1 — Query Embedding and Intent Parsing

The raw natural-language query is first parsed by the Query Parser module, which performs intent classification (factual lookup, relationship query, summarisation, comparison, or analytical reasoning), entity spotting to identify any named entities already present in the query, and temporal parsing to extract any date or time references. This structured intent representation is passed to the Retrieval Agent alongside the raw query. The query is also independently embedded into the same vector space as the document chunks, producing a 768-dimensional query vector.

Stage 2 — Parallel Retrieval: FAISS and Neo4j

The Retrieval Agent fires two queries simultaneously. The FAISS query returns the top-K document chunks (typically K=20) whose embeddings are closest to the query vector, with their cosine similarity scores and source metadata. The Neo4j query pattern-matches on any named entities identified in the parsing step, retrieves their immediate neighbourhood (all nodes within two hops), and returns the subgraph along with the source documents associated with each relationship. Both queries typically complete in under 50 milliseconds, after which the Retrieval Agent merges the result sets and assigns a composite relevance score to each candidate.

Stage 3 — Ranking, Fusion, and Context Assembly

The Reasoning Agent receives the merged candidate set and applies a multi-factor re-ranking step. Candidates are scored on semantic similarity (FAISS cosine score), graph centrality (how many relationship hops separate the candidate entity from the query entities), recency (more recent documents are boosted for time-sensitive queries), and source authority (documents tagged as authoritative receive a configurable boost). The top-N re-ranked candidates (typically

N=8–12) are then assembled into a structured context object: full text excerpts with page references, entity annotations, relationship chains, and source document metadata. The Reasoning Agent then performs a gap analysis — if the assembled context does not contain sufficient evidence to answer the query with high confidence, it formulates targeted follow-up queries and repeats the retrieval loop.

Stage 4 — LLM Reasoning, Grounding, and Response Delivery

Once the Reasoning Agent is satisfied that the context is sufficient, the assembled evidence is passed to the Synthesis Agent. This agent produces a response that is strictly grounded in the retrieved sources: every factual claim is inline-cited with document title, date, and page number, and any uncertainty is explicitly flagged. The response is then formatted according to the user's requested output mode. Before delivery, the response is checked by the Cross-Reference Agent for internal consistency and factual accuracy. Finally, the complete response — including the query, the retrieved evidence, the reasoning chain, and the final answer — is cached in Redis and GPTCache for future queries with similar embeddings. Subsequent users asking equivalent questions receive near-instantaneous responses from cache.

6. Document Lifecycle: Automated, Secure, Transparent

The document ingestion pipeline is the gateway through which all organisational knowledge enters the Nexus knowledge graph. It is deliberately designed to be fully automated for routine documents while providing mandatory human oversight for sensitive or high-stakes content. The entire pipeline is orchestrated by n8n, a self-hosted, no-code/low-code automation platform that runs locally inside the Nexus Docker environment. The sequence diagram below shows every actor, message exchange, and decision point in the pipeline.

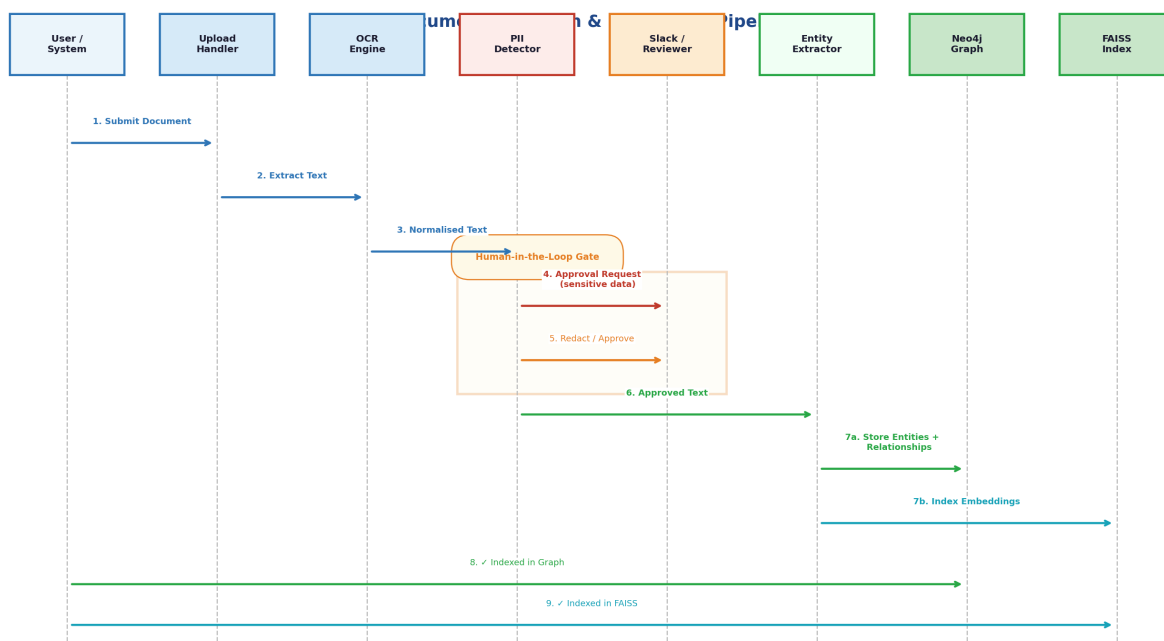


Figure 5 — Document Ingestion & Processing Pipeline: ten-stage sequence from upload through audit logging, with the human-in-the-loop approval gate highlighted

6.1 The Ten Processing Stages

Stage 1 — Document Upload and Intake

Documents enter the pipeline through any of four intake channels: direct upload via the Web UI drag-and-drop interface; API push from an external system such as a document management platform, SharePoint, or an S3-compatible object store; a watched folder monitored by an n8n workflow that triggers automatically when new files are detected; or a web scraper configured to pull content from specified URLs at scheduled intervals. Upon receipt, the intake handler assigns a unique document ID, records the upload timestamp, source channel, uploader identity, and MIME type, and places the document in an ingestion queue. Supported formats include PDF, DOCX, PPTX, XLSX, plain text, Markdown, HTML, and image files (JPG, PNG, TIFF).

Stage 2 — OCR and Text Extraction

PDF and image-based documents are processed by a locally hosted OCR engine (Tesseract 5 by default, with optional integration of more accurate commercial OCR engines for high-volume or multi-language deployments). The OCR engine produces a structured text representation that preserves page boundaries, column layout, table structure, and heading hierarchy. For native-text formats (DOCX, HTML, etc.), direct text extraction is used. The output is normalised: ligatures are decomposed, smart quotes are converted to ASCII, control characters are stripped, and duplicate whitespace is collapsed. A language detection step identifies the primary language of the document to ensure the correct embedding model and NER model are selected for subsequent stages.

Stage 3 — PII and Sensitive Content Detection

Before any content is indexed, a multi-layer content screening pass is applied. A regex-based scanner identifies structured sensitive patterns: email addresses, phone numbers, national identification numbers, credit card numbers, IBAN codes, IP addresses, API keys, private key headers, and AWS credential patterns. A complementary ML-based classifier assigns sensitivity labels — Public, Internal, Confidential, and Restricted — based on document content and metadata. Documents that trigger any sensitivity flag are held in a quarantine queue and do not proceed to indexing until a human reviewer has made an explicit decision. Non-sensitive documents pass directly to Stage 5.

Stage 4 — Human-in-the-Loop Approval Gate

This stage is the cornerstone of Nexus's governance model. When a document is held in quarantine, n8n sends a rich interactive notification to one or more designated reviewers via Slack. The notification includes the document name, upload source, detected sensitivity labels, and a preview of the flagged content with specific passages highlighted. The reviewer has three response options, each triggering a different downstream path: Approve (the document proceeds to indexing in full), Redact and Approve (the reviewer selects specific passages for removal; the redacted version proceeds to indexing), or Reject (the document is discarded; a rejection reason is logged). All decisions are recorded in the audit log with the reviewer's

identity, timestamp, and any comments provided. A configurable timeout can automatically escalate to a secondary reviewer if no decision is made within a defined window.

Stage 5 — Entity Extraction

The approved text is passed to the NER module. A transformer-based NER model (fine-tuned on domain-specific data where available) processes the text in overlapping windows and identifies spans corresponding to each of the eight supported entity types. Each span is associated with the entity's canonical form (resolved via a configurable entity resolution dictionary and fuzzy matching), its source location (document ID, page, paragraph, character offset), and a confidence score. Entities below a configurable confidence threshold are flagged for human review rather than automatically inserted into the graph.

Stage 6 — Relationship Extraction

With entities identified and disambiguated, a relation extraction module scans the text for semantic relationships between entity pairs. It uses a combination of dependency parsing (to capture syntactic relationships within a sentence) and an LLM-powered zero-shot relation classifier (to capture longer-range semantic relationships across sentences and paragraphs). The output is a set of subject-predicate-object triples, each tagged with a relationship type from the controlled vocabulary, a confidence score, and full source provenance.

Stage 7 — Knowledge Graph Population

Extracted entities and relationships are upserted into Neo4j using transactional Cypher MERGE operations that handle deduplication automatically: if an entity with the same canonical ID already exists, its properties are updated and new relationships are appended rather than duplicates being created. A post-insertion consistency check verifies that all relationship endpoints exist as nodes before committing the transaction. The graph is indexed on entity label, canonical ID, and source document ID for fast retrieval.

Stage 8 — Vector Embedding and FAISS Indexing

In parallel with Stage 7, the approved text is chunked and embedded. The chunking strategy uses a sentence-aware splitter that avoids breaking sentences mid-way, with a configurable chunk size (default 512 tokens) and overlap (default 64 tokens). Each chunk is embedded using the locally-hosted sentence-transformer model, producing a dense vector that is inserted into the FAISS index along with a metadata record in PostgreSQL that maps the vector ID back to the source document, chunk sequence number, and character offset. The FAISS index is periodically serialised to disk so that it survives container restarts.

Stage 9 — Cache Priming

Following successful indexing, the most recently added document's embeddings are optionally pre-loaded into the Redis warm cache. For high-priority documents — those tagged as authoritative or expected to be frequently queried — a background worker pre-computes responses to a set of anticipated queries and stores them in GPTCache. This ensures that the first queries against a newly indexed document return at full speed without a cold-start penalty.

Stage 10 — Audit Logging and Notification

Every action across all nine preceding stages is recorded in an immutable audit log stored in PostgreSQL. Each log entry captures the stage name, timestamp, acting user or system identity, document ID, input and output checksums, and any errors or warnings. The audit log is

append-only by design: no existing entry can be modified or deleted through the application interface. Once the pipeline completes successfully, the submitting user receives a confirmation notification (via the Web UI, email, or Slack) summarising the ingestion outcome: number of entities extracted, number of relationships added, number of FAISS vectors inserted, and total processing time.

7. Technology Stack

Every technology in the Nexus stack was chosen to satisfy three criteria: production-grade reliability, full support for on-premises deployment with no mandatory cloud connectivity, and active open-source community support that ensures long-term maintainability. The following sections describe each component in detail.

7.1 Frontend — Next.js 15, Tailwind CSS 4, Framer Motion

The Nexus web interface is built on Next.js 15, the latest major release of Vercel's React framework. Next.js provides hybrid server-side rendering (SSR) and static site generation (SSG), meaning that the initial page load is served as fully-rendered HTML from the server — eliminating the blank-screen flash of client-side-only React applications and improving both perceived performance and SEO. The App Router architecture introduced in Next.js 13 and matured in version 15 enables fine-grained control over data fetching strategies, streaming responses, and React Server Components, which significantly reduce the JavaScript bundle size sent to the browser.

Tailwind CSS 4 provides the styling system. Unlike traditional CSS frameworks that ship pre-designed components, Tailwind is a utility-first framework that composes styles from low-level utility classes. This approach produces CSS bundles that contain only the styles actually used in the application (via its JIT compiler), resulting in sub-10 KB stylesheets in production. Tailwind 4 introduces native cascade layer support and first-class dark-mode utilities that Nexus uses for its dual light/dark theme.

Framer Motion powers all animations and interactive transitions in the UI. The Neural Topology graph visualiser uses Framer Motion in combination with D3.js to animate the live knowledge graph — nodes and edges appear, merge, and rearrange fluidly as new documents are indexed or as the user explores different subgraphs. Framer Motion's spring physics engine ensures that all transitions feel natural and responsive rather than mechanical.

7.2 Backend — FastAPI, Python 3.11, SQLAlchemy

FastAPI was selected as the backend framework for three primary reasons. First, its native support for Python's asyncio runtime means that I/O-bound operations — database queries, LLM API calls, FAISS searches — do not block the event loop, allowing a single worker process to handle hundreds of concurrent requests. Second, FastAPI automatically generates OpenAPI 3.0 documentation from Python type annotations, providing a self-documenting, immediately usable API specification for integration partners. Third, FastAPI's Pydantic-based request validation provides strict runtime type checking with clear, actionable error messages, which significantly reduces the surface area for injection attacks and malformed input errors.

Python 3.11 brings significant performance improvements over earlier versions — the CPython interpreter is approximately 10–60% faster on benchmarks relevant to Nexus workloads,

including JSON serialisation, regex execution, and async I/O scheduling. SQLAlchemy 2.0 provides the ORM layer for all relational data access, using its modern 2.0-style session API that is fully compatible with asyncio. Alembic manages database migrations, ensuring that schema changes can be applied reliably across deployments without downtime.

7.3 Knowledge & Data Layer

Neo4j — Property Graph Database

Neo4j is the world's leading native graph database. Its property graph model supports labelled nodes (entity types) and typed, directed relationships (semantic predicates), both of which can carry key-value properties. The Cypher query language provides an expressive, ASCII-art-inspired syntax for describing graph patterns — for example, `MATCH (org:Organisation)-[:REGULATED_BY]->(reg:Regulation)` returns all organisations and the regulations that govern them. Neo4j's native graph storage engine follows physical pointers between nodes, making multi-hop traversals orders of magnitude faster than equivalent relational joins. Nexus uses the community edition for single-node deployments and Neo4j Enterprise with causal clustering for high-availability production environments.

FAISS — Billion-Scale Vector Search

Meta AI's FAISS library is the industry standard for high-performance dense vector similarity search. Nexus uses the IndexIVFPQ variant, which combines an inverted file index (for fast approximate nearest-neighbour search) with product quantisation (for memory-efficient vector compression). This configuration supports corpora of hundreds of millions of vectors on a single server with 64 GB of RAM, achieving average query latency of 5–15 milliseconds at 95% recall accuracy. For GPU-equipped servers, FAISS's CUDA backend reduces this latency to under 1 millisecond.

Redis — In-Memory Cache and Session Store

Redis serves as the low-latency cache layer between the agent swarm and the persistent data stores. It caches recently computed embeddings, intermediate agent context objects, session state for ongoing multi-turn conversations, and rate-limiting counters. Redis is configured with an LRU eviction policy and persistence via append-only file (AOF) logging, ensuring that cached data survives container restarts while maintaining sub-millisecond read latency.

GPTCache — Semantic LLM-Response Caching

GPTCache is an open-source library that adds semantic-level caching on top of any LLM API. Rather than caching based on exact query string equality (as a traditional key-value cache would), GPTCache embeds each incoming query and checks whether any previously cached query has a sufficiently similar embedding (configurable cosine similarity threshold, default 0.95). If a cache hit is found, the cached response is returned without invoking the LLM at all. In production deployments handling repetitive queries — such as a compliance team running daily checks against the same policy documents — GPTCache consistently achieves cache hit rates of 40–70%, reducing LLM inference costs and response latency proportionally.

7.4 Automation, Orchestration, and DevOps

n8n — Local Workflow Automation

n8n is a self-hosted, source-available workflow automation platform that provides a visual node-based editor for building complex automation pipelines without writing code. In Nexus, n8n orchestrates the entire document ingestion pipeline (Section 6), manages the Slack approval workflow, sends notification alerts when the knowledge graph surpasses configurable thresholds (e.g., a sudden spike in entities of a particular type, which might indicate a batch of new regulatory documents), and executes scheduled tasks such as index defragmentation and backup verification. n8n runs entirely inside a Docker container and communicates with other Nexus services via internal network calls — no external connectivity is required.

Docker and Kubernetes

Every Nexus service runs in a Docker container defined by a version-controlled Dockerfile. Docker Compose is provided for single-server deployments and development environments — a single "docker-compose up" command starts the entire platform. For production multi-node deployments, Kubernetes Helm charts are provided that define Deployments, Services, ConfigMaps, Secrets, PersistentVolumeClaims, HorizontalPodAutoscalers, and NetworkPolicies for every component. Kubernetes manages container scheduling, health checking, rolling updates, and self-healing restarts automatically, reducing operational overhead significantly.

Prometheus, Grafana, and ELK Stack

Nexus ships with a pre-configured observability stack. Prometheus scrapes metrics from every service (query latency percentiles, agent execution times, FAISS search latency, Neo4j query times, cache hit rates, ingestion queue depth, and error rates) at 15-second intervals. Grafana provides pre-built dashboards for operational monitoring, capacity planning, and SLA reporting. The ELK Stack (Elasticsearch, Logstash, Kibana) aggregates structured logs from all containers into a centralised, searchable store, enabling rapid root-cause analysis during incidents.

8. Deployment & Infrastructure Architecture

Nexus is designed to run entirely within an organisation's own infrastructure — on-premises servers, a private data centre, or a private cloud VPC. The architecture diagram below shows the full deployment topology, including the Kubernetes cluster layout, persistent storage volumes, load balancing, and the monitoring and observability stack.

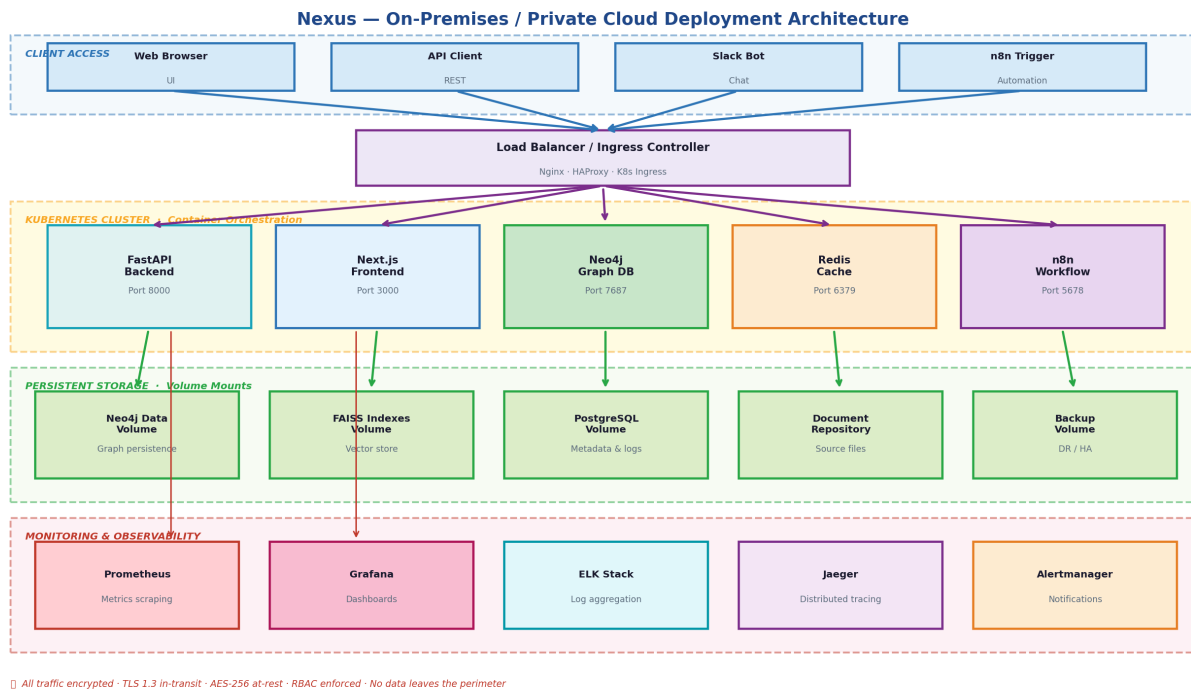


Figure 6 — Deployment Architecture: Kubernetes cluster, containerised microservices, persistent storage volumes, load balancing, and monitoring infrastructure

8.1 Container Topology

The Nexus Kubernetes deployment defines the following pods and their resource profiles:

Container	CPU Request	Memory Request	Notes
nexus-api (FastAPI)	2–4 cores	4–8 GB	Horizontally scalable; HPA on CPU > 70%
nexus-frontend (Next.js)	0.5–1 core	1–2 GB	Stateless; cacheable assets; CDN-static
neo4j	4–8 cores	16–32 GB	Single pod; Enterprise for clustering
redis	1–2 cores	4–8 GB	Redis Cluster for HA; AOF persistence
n8n	1–2 cores	2–4 GB	Single instance; SQLite or Postgres backend
embedding-worker	2–4 cores	8–16 GB	GPU-accelerated recommended for large corpora

8.2 Infrastructure Sizing Guidelines

- **Small deployment (up to 100K documents)** — 16-core server, 64 GB RAM, 2 TB NVMe SSD. Single-node Kubernetes (k3s) with all services co-located.
- **Medium deployment (100K–1M documents)** — 3-node Kubernetes cluster, 32 cores and 128 GB RAM per node, 10 TB distributed storage. Neo4j Enterprise with read replicas.
- **Large deployment (1M+ documents)** — 5+ node cluster, GPU nodes for embedding acceleration, Neo4j causal cluster, FAISS index sharded across multiple nodes, dedicated Redis Cluster.
- **Network** — Gigabit LAN minimum between cluster nodes; 10 GbE recommended for large deployments. Nexus requires no external internet connectivity once deployed.

8.3 Security Hardening

- **Network Policies** — Kubernetes NetworkPolicies enforce microsegmentation: each pod only accepts traffic from the specific pods it needs to communicate with. The database tier is completely unreachable from the public network.
- **Secrets Management** — all credentials (Neo4j passwords, JWT signing keys, embedding API keys) are stored in Kubernetes Secrets, optionally backed by HashiCorp Vault for enterprise key rotation policies.
- **Image Scanning** — all container images are scanned with Trivy during the CI/CD build process. Images with critical CVEs are blocked from deployment.
- **Pod Security Standards** — all Nexus pods run with restricted Pod Security Standards: no privilege escalation, no host network or PID namespace access, read-only root filesystems where possible.
- **TLS Everywhere** — all inter-service communication uses mutual TLS. Ingress is terminated at the Nginx or HAProxy load balancer with TLS 1.3; certificates are managed by cert-manager.

8.4 Scaling Strategies

Nexus scales both vertically (larger nodes) and horizontally (more nodes). The FastAPI gateway tier scales horizontally via Kubernetes HorizontalPodAutoscaler, which adds replicas when CPU utilisation exceeds 70% and removes them when it drops below 40%. The embedding worker tier scales similarly, with GPU availability as an additional scheduling constraint. Neo4j scales through its causal clustering feature, which replicates the graph across multiple nodes with a configurable read/write split — read-heavy query workloads are served from read replicas while write-heavy ingestion workloads target the leader. FAISS indexes can be sharded across multiple worker nodes using a consistent hashing scheme, with the Retrieval Agent querying all shards in parallel and merging the results.

9. Enterprise Use Cases

Nexus is purpose-built for environments where knowledge is complex, sensitive, and interconnected. The following use cases illustrate the qualitative difference that multi-agent GraphRAG makes compared to conventional document search or flat-RAG approaches.

9.1 Financial Services — Regulatory Compliance and Risk

Policy Gap Analysis

A compliance officer asks: "Which of our trading desk procedures are not covered by the updated MiFID II guidelines published last month?" Nexus traverses the graph from the regulatory document to the procedures it references, identifies which procedures do not have a corresponding compliance mapping, and returns a structured gap report with direct links to the relevant regulatory passages. This analysis, which would take a compliance team days to perform manually, completes in seconds.

Counterparty Risk Propagation

A risk analyst asks: "Which of our active positions have indirect exposure to Counterparty X through shared collateral or custody arrangements?" The graph query traverses three relationship hops — position → custodian → collateral pool → counterparty — returning the full exposure chain for each affected position.

Regulatory Change Impact

When a new regulation is ingested, Nexus automatically identifies all contracts, procedures, and products in the knowledge graph that cite the superseded regulation and flags them for review, with a prioritised remediation list ordered by business impact.

9.2 Supply Chain and Manufacturing

Multi-Tier Supplier Risk Mapping

A procurement director asks: "Which of our Tier-2 and Tier-3 suppliers are geographically concentrated in a single region?" The graph traverses SUPPLIED_BY edges two and three hops from the organisation node and clusters suppliers by LOCATED_IN relationships, revealing concentration risk that is invisible in any single procurement document.

Specification Traceability

An engineering manager asks: "Which product lines are affected if Supplier Y cannot deliver Component Z?" The graph traverses USES_COMPONENT → MANUFACTURED_BY → PRODUCT_LINE relationships, returning the full impact chain across the bill of materials and production schedule.

9.3 Healthcare and Life Sciences

Clinical Literature Synthesis

A medical researcher asks: "What does the literature published in the last two years say about the interaction between Drug A and Drug B in elderly patients with renal impairment?" Nexus fuses FAISS semantic retrieval (to find papers discussing this drug combination) with graph traversal (to identify papers that cite the same clinical trials or share author affiliations), returning a synthesised review with full citations and highlighted consensus and disagreement points.

Adverse Event Pattern Detection

By connecting adverse event reports, drug approval documents, and clinical trial results in the knowledge graph, Nexus can surface emerging signal patterns — e.g., a cluster of adverse

event reports sharing a common concomitant medication — that would be undetectable by querying each document type in isolation.

HIPAA-Safe Local Processing

Because all processing occurs on-premises with no data leaving the hospital's network, Nexus is inherently compatible with HIPAA Privacy Rule requirements for PHI handling. The human approval gate provides the additional safeguard of preventing patient-identifiable information from entering the knowledge graph unless explicitly authorised by a designated data steward.

9.4 Legal and Professional Services

Contract Intelligence

A legal team asks: "Which of our active contracts contain change-of-control clauses that would be triggered by the proposed acquisition of Company X?" Nexus traverses CONTRACT → CONTAINS_CLAUSE → TRIGGERED_BY relationships and returns the relevant contract clauses with their exact text, parties, and governing law, enabling lawyers to prioritise their review effort.

Precedent Discovery

A litigator asks: "In which prior matters did we successfully argue that limitation clause Y was unenforceable under English law?" The knowledge graph connects past case matter records to the specific arguments made and their outcomes, enabling rapid precedent retrieval across decades of institutional knowledge that would otherwise require hours of manual document review.

9.5 Research and Development

Patent Landscape Mapping

An IP team asks: "Which organisations hold patents that are likely to block the commercialisation of Technology X, and what is the citation network connecting those patents to our own portfolio?" The graph traverses patent citation relationships to surface freedom-to-operate risks and identify potential licensing partners or acquisition targets.

Cross-Disciplinary Discovery

A research director asks: "Which external researchers have published work that combines Technique A with Application B?" Nexus identifies the intersection of two research communities in the knowledge graph — researchers who publish on Technique A connected to those who publish on Application B via shared citations, co-authorship, or institutional affiliation.

10. Comparative Analysis

The table below compares Nexus against three alternative approaches: traditional keyword search (e.g., Elasticsearch), standard RAG (a single LLM with a flat vector index), and a cloud-based LLM API service. The comparison focuses on the dimensions that matter most for enterprise knowledge retrieval.

Dimension	Keyword Search	Standard RAG	Nexus
Multi-hop reasoning	Not possible	Very limited	Full graph traversal

			— unlimited hops
Relationship awareness	None	Implicit only	Explicit typed relationships in Neo4j
Answer grounding	Keyword match only	Chunk-level citation	Entity + document + page-level citation
Data sovereignty	Possible (local)	Possible (local)	Guaranteed — fully on-premises
Human governance	None	None built-in	Built-in approval gate + audit trail
Scalability	Excellent (ES native)	Limited by context window	Unlimited via K8s + FAISS sharding
Compliance posture	Neutral	Depends on deployment	GDPR · HIPAA · SOC 2 · NIST CSF
Query latency (p99)	< 100 ms	2–10 s (LLM call)	< 500 ms (cached) / 3–8 s (uncached)

11. Governance, Security, and Compliance

11.1 Data Governance Framework

Nexus implements a comprehensive data governance framework built on four principles: provenance, control, transparency, and accountability. Every entity and relationship in the knowledge graph is tagged with the source document that originated it, the extraction timestamp, the model version that performed the extraction, and the confidence score. This provenance chain means that when an answer is delivered to a user, every factual claim in that answer can be traced back to its originating sentence in its originating document — a capability that auditors and regulators increasingly require.

- **Document Lineage** — every entity in the graph maintains a complete list of all documents that reference it, enabling administrators to understand the full scope of impact before deleting or modifying a document.
- **Version Control** — every version of every ingested document is retained in an immutable store. When a document is updated, the new version is indexed alongside the old one, with graph relationships annotated to indicate which version they were extracted from.
- **Right to Erasure (GDPR Article 17)** — a dedicated data deletion workflow allows administrators to remove a document from the system completely: the source file is deleted, all FAISS vectors derived from it are removed, all graph nodes and relationships tagged with its document ID are deleted, and the deletion event is recorded in the audit log.
- **Data Retention Policies** — configurable retention rules can automatically expire documents (and their derived knowledge graph data) after a specified period, ensuring compliance with records management policies.

11.2 Access Control and Authentication

Nexus implements role-based access control (RBAC) at multiple granularity levels. At the system level, four default roles are defined: Viewer (read-only query access), Contributor (can upload documents; queries subject to standard controls), Reviewer (can approve or reject documents in the governance queue), and Administrator (full system access including

configuration, user management, and audit log access). At the data level, document-level access controls allow specific documents to be restricted to named users or groups, ensuring that sensitive materials are only accessible to authorised personnel even after indexing. JWT-based authentication with configurable token expiry integrates with enterprise identity providers via SAML 2.0 and OpenID Connect (supporting Microsoft Entra ID, Okta, and others).

11.3 Security Controls

- **Encryption at Rest** — all database volumes (Neo4j, PostgreSQL, document repository) are encrypted with AES-256. LUKS disk encryption can be applied at the OS level for additional assurance.
- **Encryption in Transit** — all internal inter-service communication uses mutual TLS 1.3. All external API calls use TLS 1.3 with strong cipher suites; TLS 1.0 and 1.1 are disabled.
- **Secret Rotation** — database credentials, JWT signing keys, and API keys are stored in Kubernetes Secrets with optional HashiCorp Vault integration for automated rotation on a configurable schedule.
- **Vulnerability Management** — container images are scanned with Trivy at build time and runtime. Critical CVEs trigger automated alerts and block deployment until patched.
- **Network Microsegmentation** — Kubernetes NetworkPolicies restrict pod-to-pod traffic to explicitly authorised flows. The database tier accepts connections only from the API tier; n8n accepts connections only from designated trigger sources.
- **Penetration Testing** — Nexus is designed to be tested under standard enterprise penetration testing methodologies. OWASP Top 10 mitigations are applied throughout the API layer.

11.4 Regulatory Compliance Alignment

Nexus's architecture is designed to support compliance with the following regulatory frameworks. Achieving certification or attestation under any of these frameworks requires organisational processes in addition to technical controls, but Nexus provides the technical foundation on which those processes can be built.

- **GDPR (EU General Data Protection Regulation)** — data minimisation by design, right to erasure workflows, data processing records, and audit trails satisfy Articles 5, 17, 30, and 32.
- **HIPAA (Health Insurance Portability and Accountability Act)** — PHI isolation via human approval gates, access controls, encryption at rest and in transit, and audit logging satisfy the Technical Safeguard requirements of the HIPAA Security Rule.
- **SOC 2 Type II** — the immutable audit log, RBAC controls, change management workflows, and monitoring infrastructure provide evidence for the Security, Availability, and Confidentiality trust service categories.
- **NIST Cybersecurity Framework** — Nexus maps to all five NIST CSF functions: Identify (asset inventory, data classification), Protect (access control, encryption, training), Detect (monitoring, anomaly detection), Respond (incident workflows), and Recover (backup and restore procedures).
- **ISO 27001** — the access control, incident management, and audit logging capabilities align with the ISO 27001 Annex A controls for information security management.

12. Performance Characteristics and Benchmarks

Performance in a system like Nexus is multi-dimensional: raw retrieval speed, end-to-end query latency (including LLM inference), throughput under concurrent load, and ingestion rate all matter depending on the use case. The following figures are representative of a medium-sized deployment (500K documents, 3-node Kubernetes cluster, CPU-only inference).

Metric	Value	Notes
FAISS p50 query latency	< 8 ms	Single-node CPU, 50M vectors
FAISS p99 query latency	< 25 ms	Single-node CPU, 50M vectors
Neo4j 3-hop query latency	15–80 ms	Depends on subgraph size
End-to-end (cached)	80–200 ms	GPTCache hit, no LLM call
End-to-end (uncached)	3–8 seconds	Full agent swarm + LLM inference
Ingestion throughput	200–500 pages/min	OCR + NER + embedding + indexing
Concurrent queries (p95)	50 req/s	3 FastAPI replicas, cached mix

GPU acceleration of the embedding worker (using an NVIDIA A10 or equivalent) increases ingestion throughput by approximately 4–6x and reduces FAISS query latency to under 1 millisecond by enabling GPU-side index search. For latency-sensitive deployments, deploying a quantised LLM (e.g., a 4-bit AWQ-quantised Mistral-7B) locally reduces uncached query latency to 1–2 seconds while maintaining response quality comparable to hosted API models for most knowledge retrieval tasks.

13. Future Roadmap

The Nexus roadmap is driven by three strategic themes: expanding the reasoning capability of the agent swarm, deepening real-time and streaming data support, and broadening the ecosystem of enterprise integrations.

13.1 Advanced Reasoning Capabilities

- **Causal Inference Engine** — a dedicated reasoning module that models cause-effect relationships extracted from documents, enabling queries such as "What are the likely downstream effects of discontinuing Product X?" based on inferred causal chains in the knowledge graph.
- **Counterfactual and Scenario Analysis** — building on the causal graph, a scenario simulation capability will allow users to pose "what-if" questions and receive graph-based impact assessments rather than speculative LLM outputs.
- **Temporal Reasoning** — explicit support for time-series relationships in the graph (PRECEDED_BY, SUPERSEDED_BY, CO-OCCURRED_AT) will enable richer temporal queries such as trend analysis across document corpora.
- **Multi-Modal Knowledge Extraction** — extending entity and relationship extraction to handle tables, charts, diagrams, and images within documents, enabling Nexus to ingest the full information content of complex technical documents.

13.2 Real-Time and Streaming Data

- **Kafka Integration** — a streaming ingestion connector will allow Nexus to consume document events from Apache Kafka topics in real time, enabling sub-second knowledge graph updates as new documents are published.
- **Event-Driven Alerts** — configurable alert rules based on knowledge graph state changes (e.g., "alert me when a new regulation affecting Financial Instrument Type X is ingested") will be delivered via Slack, email, or webhook.
- **Live Data Source Connectors** — pre-built connectors for SharePoint, Confluence, Notion, Google Drive, and S3-compatible object stores will enable automatic, incremental synchronisation of external document repositories.

13.3 Extended Integrations

- **Salesforce and HubSpot** — CRM integration that enriches the knowledge graph with customer and opportunity data, enabling relationship-aware queries across sales and knowledge domains.
- **SAP and Oracle ERP** — connectors for supply chain, procurement, and financial data that allow the knowledge graph to bridge structured ERP records with unstructured document knowledge.
- **Jupyter Notebook SDK** — a Python SDK for data scientists and analysts to query the Nexus knowledge graph programmatically, enabling integration with machine learning workflows and data pipelines.
- **Microsoft 365 Copilot Plugin** — a Copilot plugin that surfaces Nexus answers directly within Word, Excel, and Teams, eliminating context-switching for knowledge workers already in the Microsoft ecosystem.

13.4 Platform Maturity

- **Fine-Tuning Pipeline** — an integrated workflow for fine-tuning the NER and relation extraction models on organisation-specific entity types and domain vocabulary, improving extraction accuracy for specialised corpora.
- **Multi-Tenancy** — support for logical data isolation between business units or clients within a single Nexus deployment, enabling multi-tenant SaaS deployments with per-tenant access controls and billing.
- **GraphQL API** — a GraphQL interface over the knowledge graph will enable frontend developers to write expressive, type-safe queries without learning Cypher.

14. Conclusion

The enterprise AI landscape is littered with solutions that work well in demos and disappoint in production. Flat-RAG systems answer easy questions adequately but fail at the hard ones. Cloud-based LLM APIs offer impressive general capability but at the cost of data sovereignty and governance control. Traditional knowledge management platforms provide structure but lack the intelligence to synthesise insight from unstructured content.

Nexus is built to close all three gaps simultaneously. Its multi-agent swarm delivers reasoning depth that single-call LLM architectures cannot match. Its GraphRAG architecture makes the full relational complexity of organisational knowledge queryable for the first time. Its local-first

deployment model ensures that data sovereignty is not a feature to be unlocked at enterprise tier — it is the default. And its human-in-the-loop governance framework ensures that the knowledge graph remains accurate, authorised, and auditable at every step.

For organisations ready to move beyond keyword search and shallow RAG, and into a world where every question — no matter how complex, multi-hop, or relationship-rich — can be answered with precision, citation, and confidence, Nexus provides the infrastructure to make that transition today.